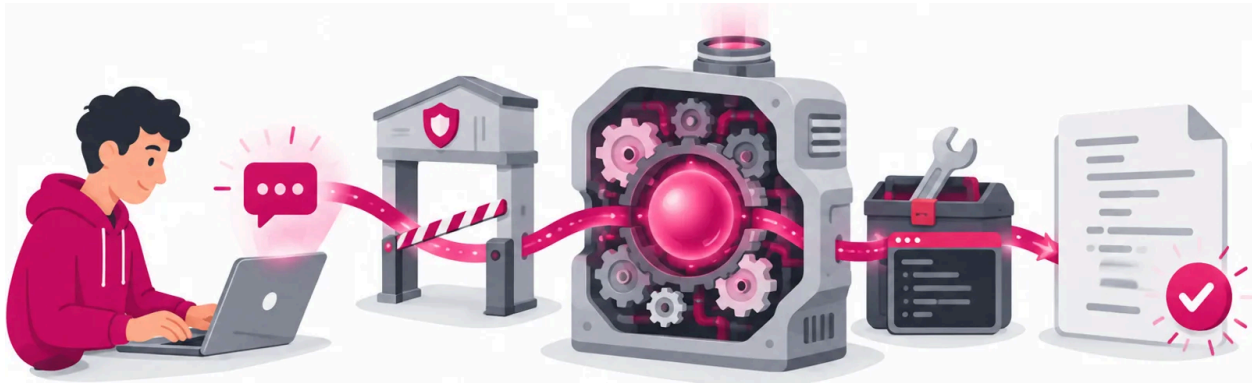
ZENIKA · FORMATION IA · 2H

# Sous le capot

*LLM, SLM, agents, harness, orchestrateurs : comment ça marche vraiment*

Du prompt tapé dans un agent de code jusqu'au fichier modifié sur le disque.

# À la fin de la session, vous saurez…



*Le fil rouge : suivre une requête, de votre prompt jusqu'au fichier modifié.*

**Nommer**
les six briques
PARTIE 1

**Expliquer**
le moteur
PARTIE 2

**Décortiquer**
la boucle d'agent
PARTIE 3

**Choisir**
harness et orchestrateur
PARTIE 4

**Sécuriser**
guardrails et gateways
PARTIE 5

© ZENIKA 2026 | All rights reserved

# Agenda

1. Ouverture — 5 min
2. Cartographie — 20 min
3. LLM & SLM sous le capot — 35 min
4. Anatomie d’un agent — 20 min
5. Harness & orchestrateurs — 20 min
6. Guardrails & gateways — 15 min
7. Synthèse — 5 min

© ZENIKA 2026 | All rights reserved

Partie 1 · 20 min

# Cartographie

Six mots qu’on mélange : une définition et des produits du marché pour chacun.




© ZENIKA 2026 | All rights reserved

# Six notions, une phrase chacune

LLM

## Le moteur

Prédit le token suivant.

GPT-5 | Claude | Gemini

SLM

## Le petit moteur

Le même, en 1 à 15 Md de paramètres.

Phi-4 | Gemma 3 | Qwen3

AGENT

## Le modèle qui agit

Un modèle dans une boucle, avec des outils.

Copilot agent | Devin | Codex

HARNESS

## Tout sauf le modèle

Le code autour du modèle.

Claude Code | Codex CLI | OpenCode

ORCHESTRATEUR

## Le chef d'orchestre

Coordonne plusieurs agents.

LangGraph | CrewAI | Spec Kit

SDD

## L'orchestrateur des specs

Préconfiguré, piloté par les specs.

Spec Kit | Kiro | OpenSpec

© ZENIKA 2026 | All rights reserved

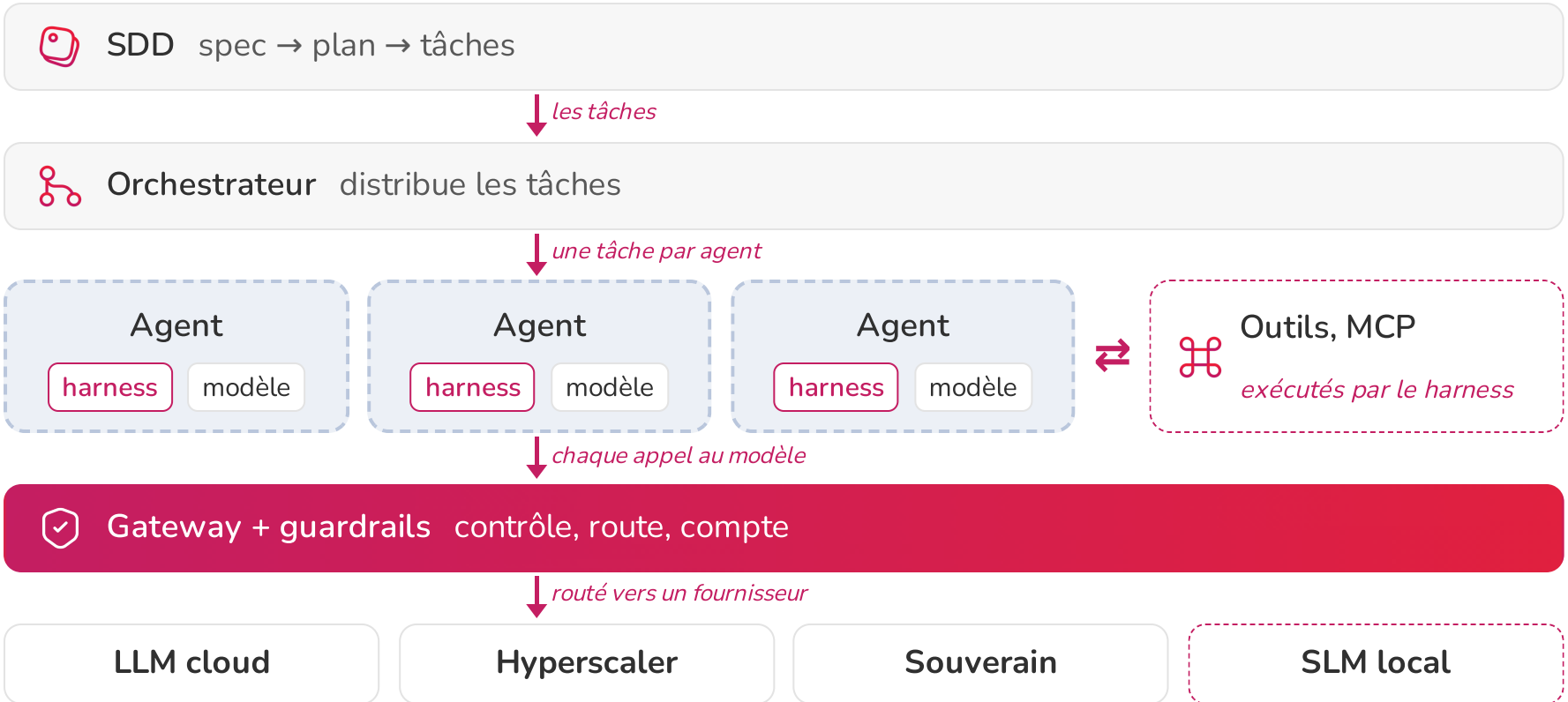
# Une analogie qui tient la route



| | | |
|---|---|---|
| **LLM / SLM** | → | le moteur |
| **Agent** | → | le véhicule |
| **Harness** | → | ses spécificités<br>grand coffre, frigo… |
| **Orchestrateur** | → | le dispatch |
| **SDD** | → | le bon de commande |

© ZENIKA 2026 | All rights reserved

# Qui parle à qui : la pile complète




© ZENIKA 2026 | All rights reserved

# Chatbot, workflow ou agent ?

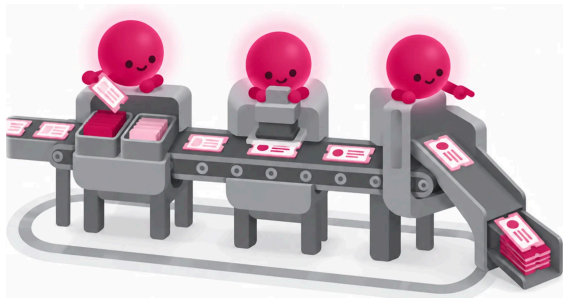
DÉTAIL ↗



## Chatbot

Un échange, un appel au modèle.

qui décide ? **l'humain**



## Workflow LLM

Des étapes fixes : classer → résumer → router.

qui décide ? **le code**



## Agent

Il choisit ses outils et quand s'arrêter.

qui décide ? **le modèle**

© ZENIKA 2026 | All rights reserved

# Harness vs orchestrateur : la frontière



HARNESS

**Un véhicule équipé**

fait d'un modèle **un agent**

`Claude Code` `Codex CLI` `Aider`



ORCHESTRATEUR

**Le dispatch**

fait travailler **plusieurs agents**

`LangGraph` `CrewAI` `Spec Kit`

© ZENIKA 2026 | All rights reserved

# Quiz : qui est qui ?

| | | | |
|---|---|---|---|
| Ollama<br>runtime local | Gemma 3 4B<br>SLM | Claude Sonnet<br>LLM | Claude Code<br>harness |
| LangGraph<br>orchestrateur | GitHub<br>Spec Kit<br>outillage SDD | LiteLLM<br>gateway | Llama Guard<br>guardrail |
| MCP<br>protocole | Copilot<br>coding agent<br>agent cloud | text-<br>embedding-3<br>embedding | vLLM<br>serveur d'inférence |

© ZENIKA 2026 | All rights reserved

Partie 2 · 35 min

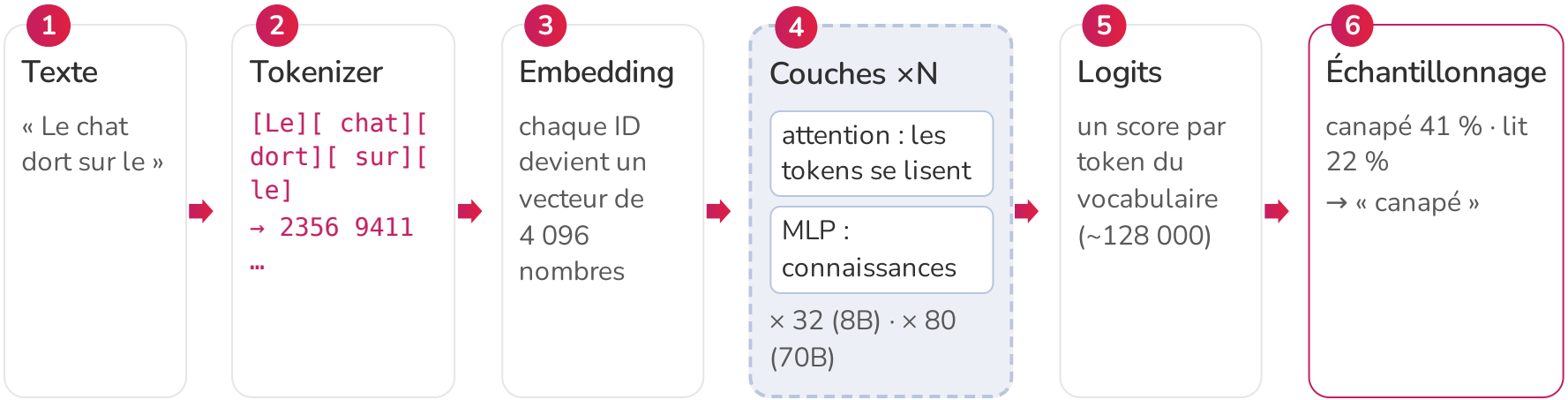
# LLM & SLM sous le capot

On suit la phrase « Le chat dort sur le » jusqu'au mot suivant.




© ZENIKA 2026 | All rights reserved

# Le trajet d’une phrase dans le modèle




**↺ boucle autorégressive : le token choisi est ajouté au texte, et on recommence**

Ordres de grandeur pour Llama 3 8B : vocabulaire 128 256 tokens · vecteurs de 4 096 dimensions · 32 couches · 8 milliards de paramètres

© ZENIKA 2026 | All rights reserved

# Tokenisation : le modèle ne lit pas des mots

| | | |
|---|---|---|
| Mot rare | `Anti` `constitution` `nellement` | 1 mot → **3 tokens** |
| Code | `def` ` get` `_user` `(id)` `:` | 4 mots → **5 tokens** |

© ZENIKA 2026 | All rights reserved

# Embeddings : du numéro au sens

le mot

**« chat »**

↓

le token

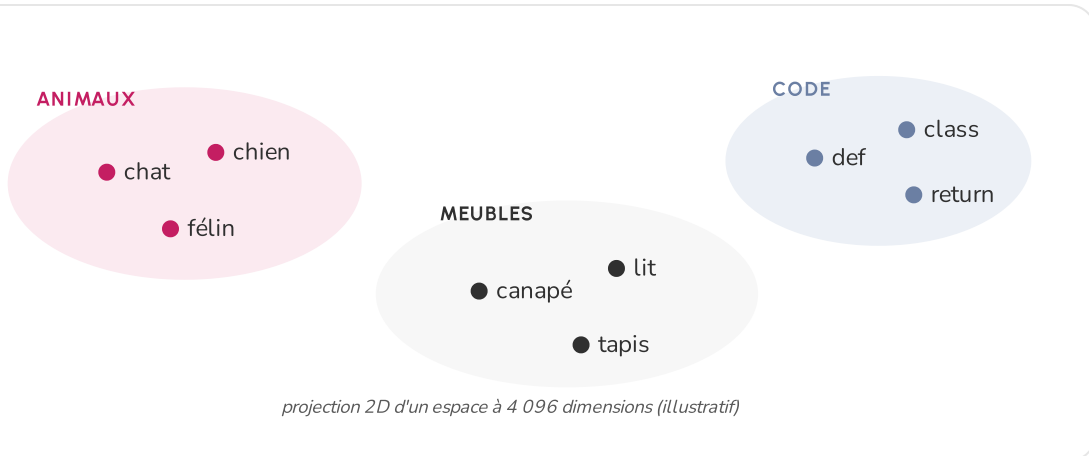
**n° 9411**

↓

le vecteur

**[0,12 · –0,48 · … ]**

4 096 nombres




*projection 2D d'un espace à 4 096 dimensions (illustratif)*

*Chaque vecteur est un point : les mots au sens proche finissent* **proches**.

© ZENIKA 2026 | All rights reserved

# Une couche transformer, c’est deux opérations

le vecteur de
**« il »**

↓

① Attention
**il regarde les mots d'avant**
et trouve « chat »

↓

② MLP
**il ajoute ce que le modèle sait**
un chat peut être fatigué

↓

on répète
**× 32 couches**

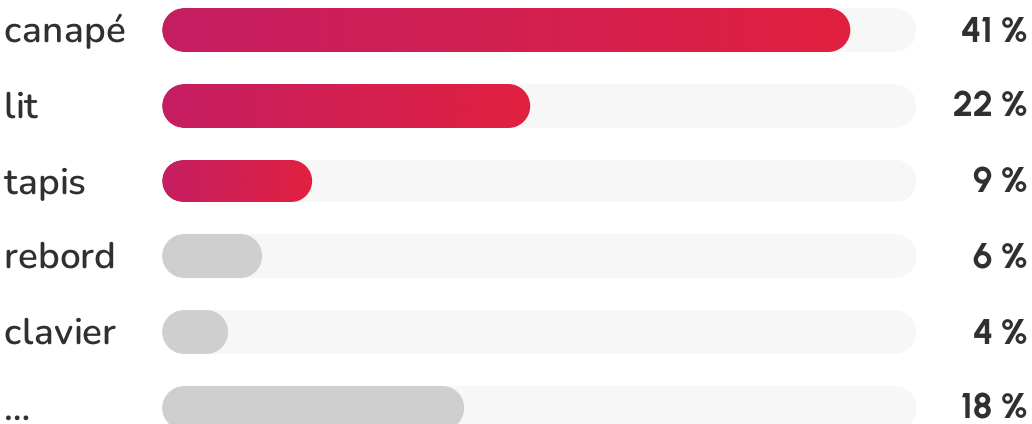



*Après les couches, « il » sait qu'il désigne le chat.*

© ZENIKA 2026 | All rights reserved

# Choisir le mot suivant : logits, température, top-p

## Après « Le chat dort sur le »




*En couleur : le noyau retenu avec top-p = 0,7.*

**Température basse**
toujours le favori

**Température haute**
plus de surprises

© ZENIKA 2026 | All rights reserved

# D’où viennent les poids : trois phases d’entraînement

1



## Pré-entraînement

Prédire le token suivant sur tout Internet.

RÉSULTAT

un modèle « de base »

2



## Fine-tuning supervisé

Des exemples de bonnes réponses.

RÉSULTAT

un modèle « instruct »

3



## Renforcement

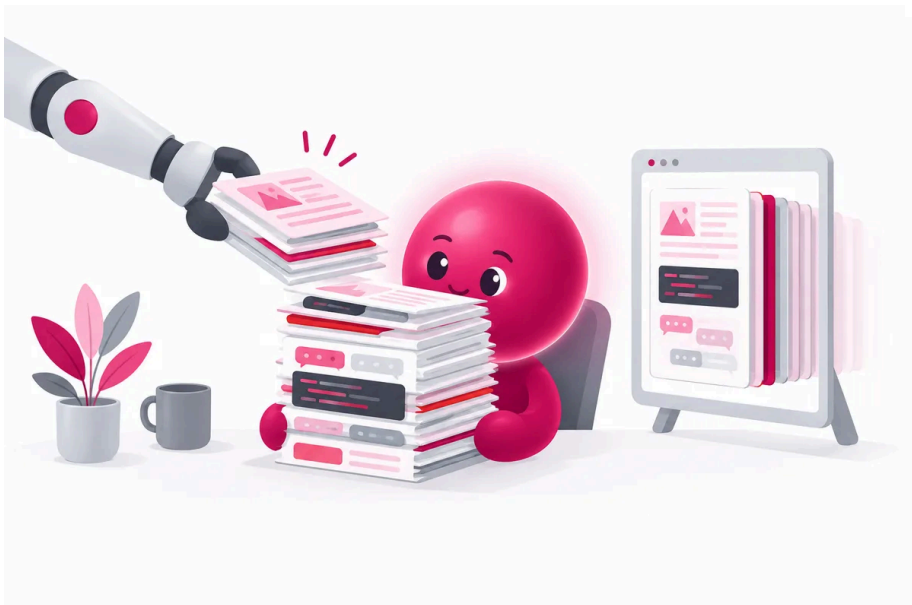
Récompenser ce qui marche : préférences, tests qui passent.

RÉSULTAT

raisonnement et agents

© ZENIKA 2026 | All rights reserved

# La context window : la seule mémoire de travail



**Le modèle n'a pas de mémoire entre deux appels : à chaque tour, le harness renvoie tout.**

**128 k**

tokens : modèles ouverts

**1 M+**

tokens : offres longues

© ZENIKA 2026 | All rights reserved

# KV cache : ne pas tout recalculer à chaque token

## 1 Prefill : lire le prompt

Tous les tokens du prompt traités en parallèle.

`Le` `chat` `dort` `sur` `le`

**KV cache : K et V de chaque token**

- limité par le calcul (GPU saturé)
- fixe le temps avant le premier token (TTFT)

## 2 Decode : écrire la réponse

Un seul nouveau token par passe.

`Le` `chat` `dort` `sur` `le` **`canapé`**

**cache réutilisé tel quel**
**+ K, V de « canapé »**

- limité par la bande passante mémoire
- fixe les tokens par seconde

© ZENIKA 2026 | All rights reserved

# Le prix du KV cache : la mémoire GPU

## Llama 3 8B en 16 bits, une conversation de 128 k tokens




*128 Ko de cache par token.*

**LES PARADES**

- GQA
- cache quantifié
- PagedAttention

© ZENIKA 2026 | All rights reserved

# Prompt caching : la facture de vos agents

DÉTAIL ↗

LE PRINCIPE

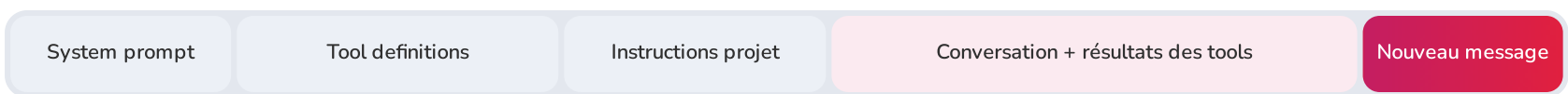
## Même début de requête = prefill sauté

Le fournisseur garde le KV cache du préfixe : réponse plus rapide, entrée **beaucoup moins chère**.

**~90 %**

de remise sur l'entrée lue en cache

DU PLUS STABLE AU PLUS VOLATIL




← *stable : servi par le cache d'un tour à l'autre*

*volatil : seule la fin est recalculée* →

© ZENIKA 2026 | All rights reserved

# Rendre un modèle plus léger : quatre techniques



### Quantification

Moins de bits par poids : 16 → 4.



### Distillation

Un petit modèle imite un grand.



### Élagage

On retire ce qui sert peu.



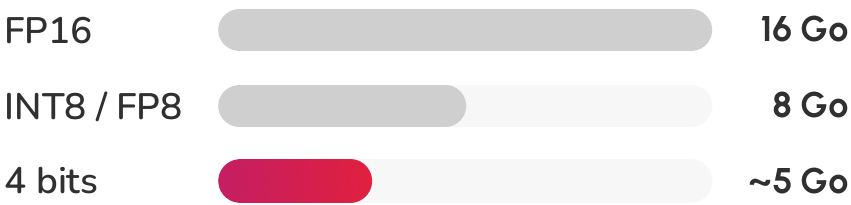
### Mixture of Experts

Seule une partie des poids travaille.

© ZENIKA 2026 | All rights reserved

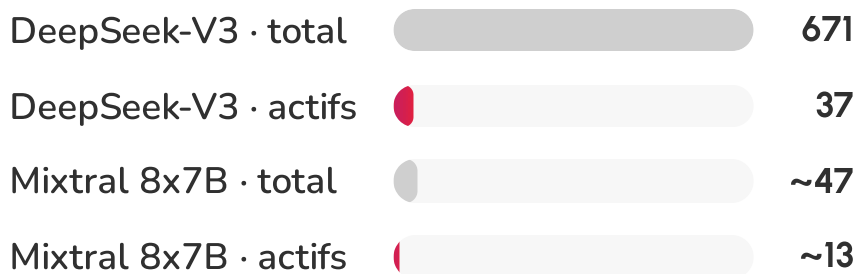
# Ce que pèse un modèle

## Quantification : mémoire d'un modèle 8B




*Mémoire des poids ≈ paramètres × octets par paramètre, plus le KV cache.*

## MoE : paramètres totaux vs actifs (milliards)

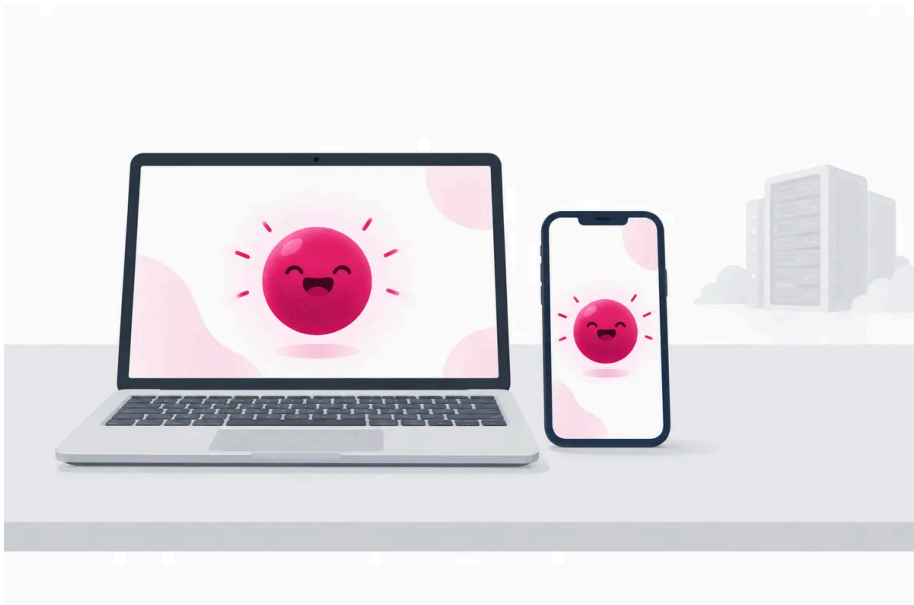



*Le calcul suit les paramètres actifs, mais tous les poids doivent tenir en mémoire.*

© ZENIKA 2026 | All rights reserved

# Les SLM : petits et locaux

DÉTAIL ↗



**1 à 15**

## milliards de paramètres

Même architecture qu'un LLM, en plus petit.

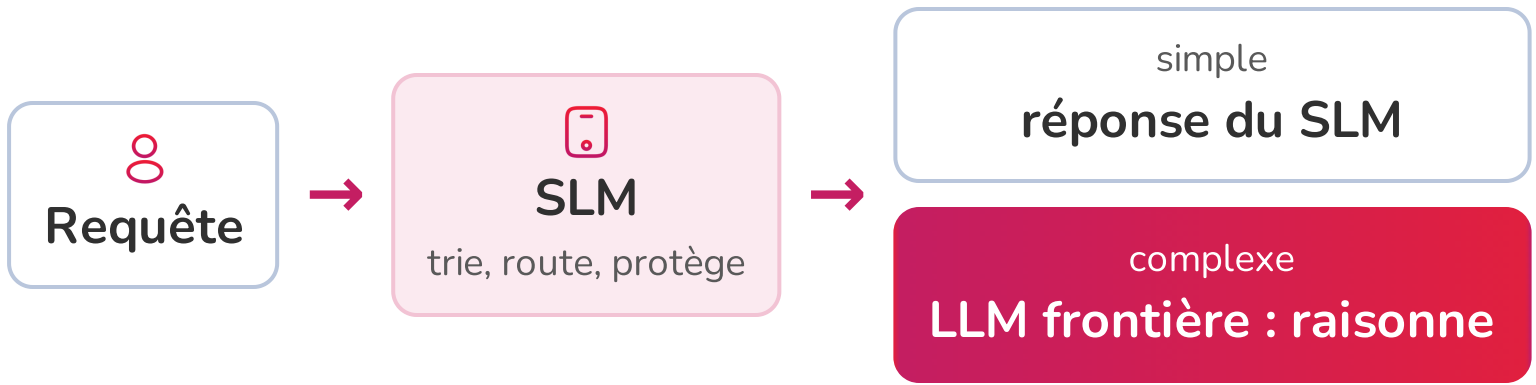
**Poste**
Ollama, MLX

**Serveur**
vLLM

**Embarqué**
Gemini Nano

© ZENIKA 2026 | All rights reserved

# SLM et LLM : le pattern hybride




**SLM** : classer, extraire, données sensibles, latence faible

**Limites** : raisonnement long, gros agents autonomes

© ZENIKA 2026 | All rights reserved

# Ce que la mécanique change pour un développeur

**TOKENS**

Comptez en tokens

**AUTORÉGRESSIF**

Demandez des sorties courtes

**SANS ÉTAT**

Tout est dans le contexte

**KV CACHE**

Gardez le préfixe stable

**PLAUSIBLE**

Vérifiez

**TAILLE**

Le bon modèle au bon endroit

© ZENIKA 2026 | All rights reserved

Partie 3 · 20 min

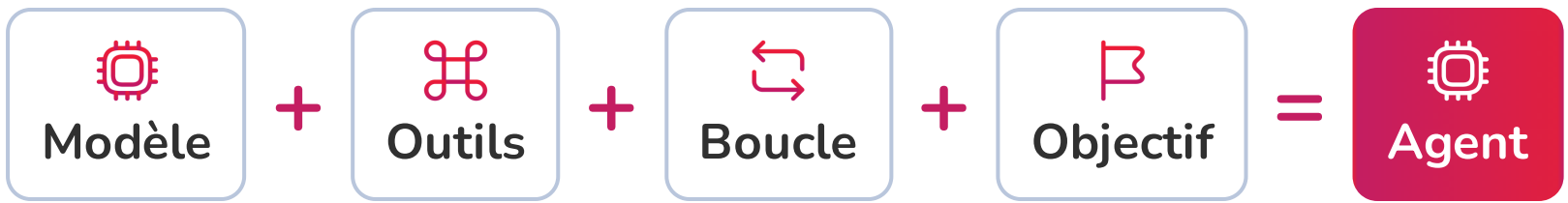
# Anatomie d’un agent

Un modèle qui n’écrit que des tokens finit par lire des fichiers, lancer des tests et ouvrir une PR.




© ZENIKA 2026 | All rights reserved

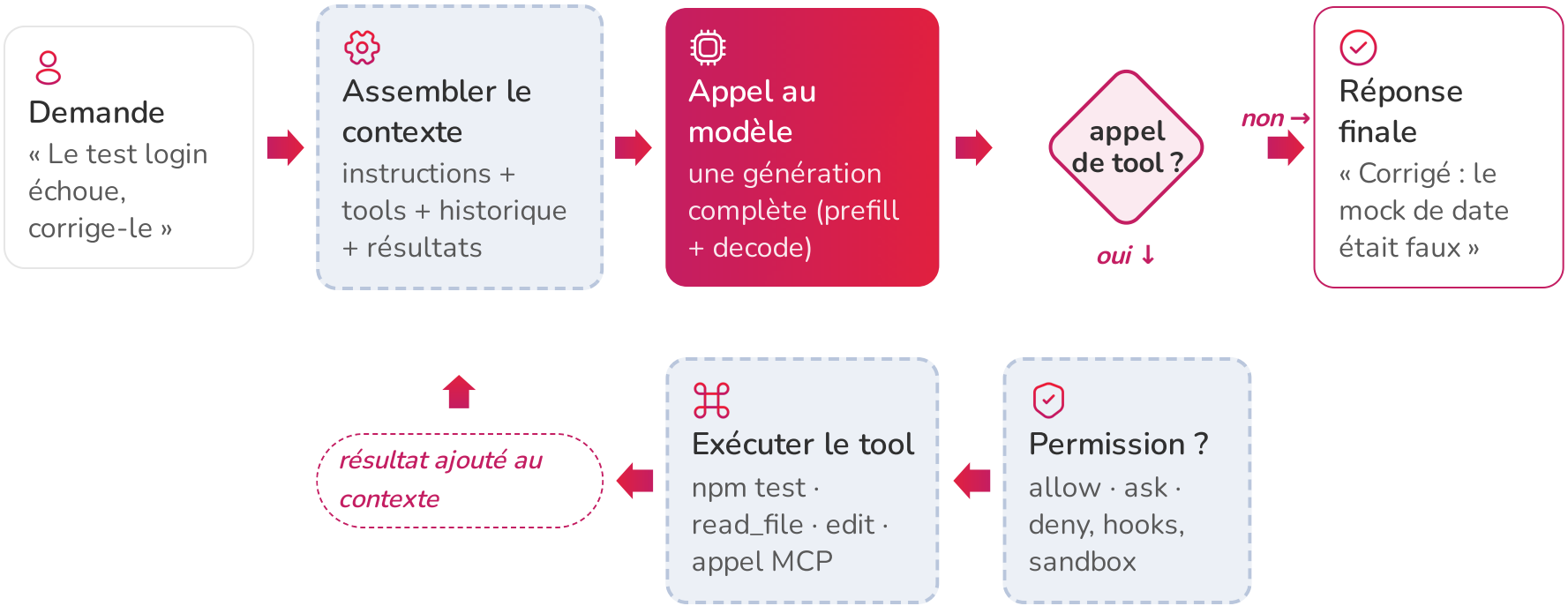
# Agent = modèle + outils + boucle + objectif




**Le modèle n'exécute rien. Il écrit une demande d'action.**

© ZENIKA 2026 | All rights reserved

# La boucle, tour par tour




Un tour = un appel modèle complet. Une tâche simple : 5 à 20 tours. Une grosse tâche : des centaines.

© ZENIKA 2026 | All rights reserved

# Tool calling : ce qui transite réellement

## ① Le harness déclare

```
{
  "name": "run_tests",
  "description": "Lance l
  "input_schema": {
    "path": "string"
  }
}
```

## ② Le modèle demande

```
{
  "type": "tool_use",
  "name": "run_tests",
  "input": {
    "path": "tests/login"
  }
}
```

## ③ Le harness répond

```
{
  "type": "tool_result",
  "content":
    "FAIL login:42
     got 401"
}
```

© ZENIKA 2026 | All rights reserved

# Mémoire : quatre étages

1



## Context window

La conversation en cours.

RÔLE

mémoire de travail

2



## Compaction

L'historique résumé.

RÔLE

résumé

3



## Fichiers

AGENTS.md, notes, plans.

RÔLE

mémoire persistante

4



## Recherche

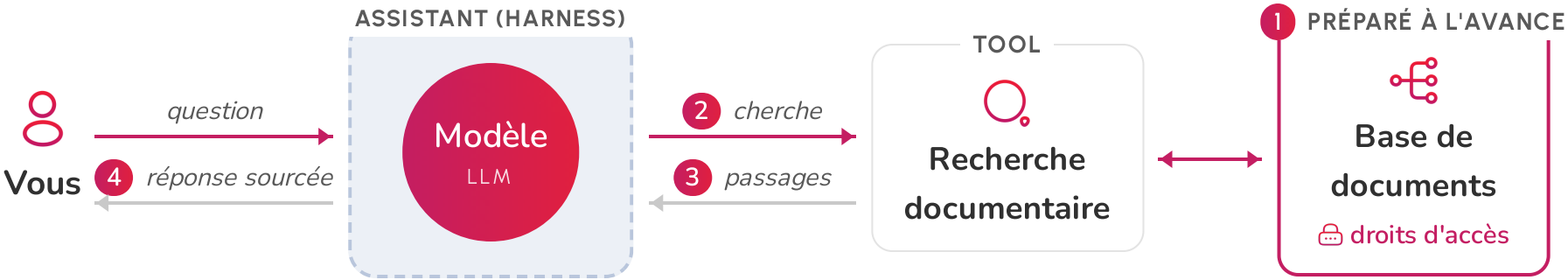
grep, RAG, MCP.

RÔLE

mémoire à la demande

© ZENIKA 2026 | All rights reserved

# RAG : le retour des embeddings




*Exemple : « Comment configure-t-on le retry du client HTTP interne ? »*

**1**

### Indexer *(indexing)*

Un vecteur par morceau de doc.

> *Wiki et ADR dans pgvector*

**2**

### Chercher *(retrieval)*

Les morceaux les plus proches de la question.

> *3 passages sur le retry*

**3**

### Augmenter *(augmentation)*

Ils entrent dans le contexte.

> *3 passages, pas 400 pages*

**4**

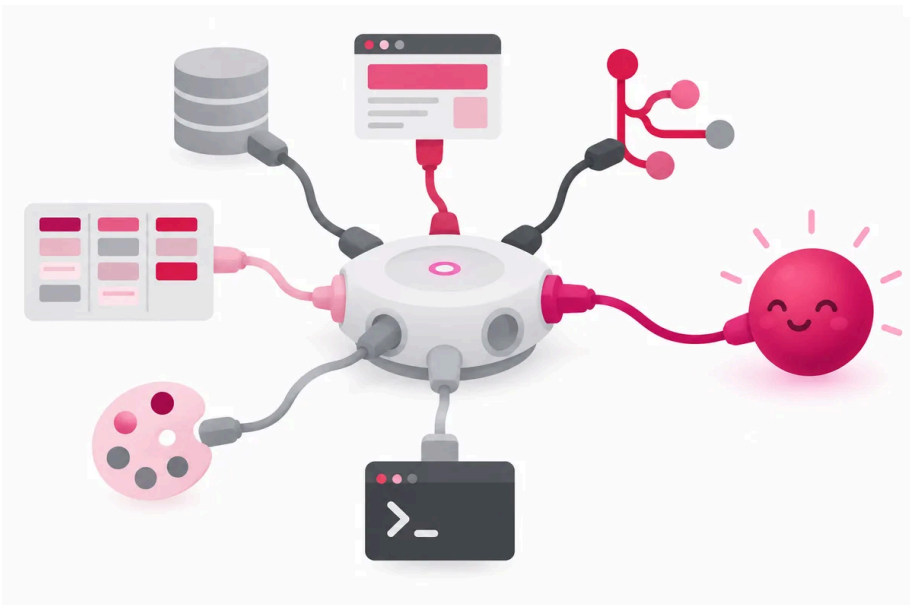
### Générer *(generation)*

Une réponse qui cite ses sources.

> *« maxRetries = 3 [ADR-012] »*

© ZENIKA 2026 | All rights reserved

# MCP : une prise standard pour les outils



MODEL CONTEXT PROTOCOL

## L'« USB-C » des agents : un serveur écrit une fois, branché dans tous les harness.

**Tools**
des actions

**Resources**
des données

**Prompts**
des modèles

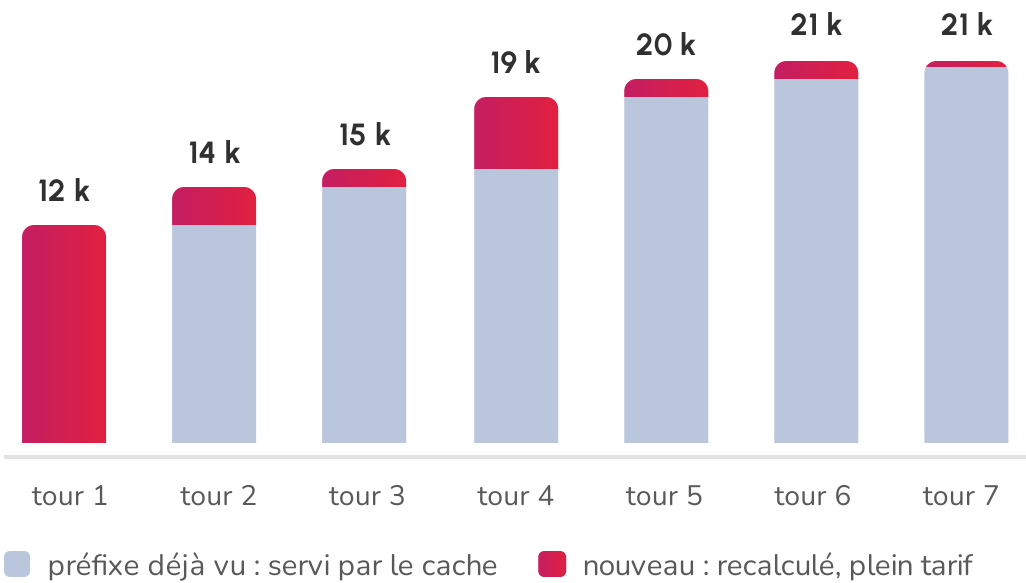
© ZENIKA 2026 | All rights reserved

# Une trace réelle, commentée

| Tour | Le modèle demande | Le harness renvoie | Contexte |
|---|---|---|---|
| 1 | `bash: npm test -- login` | échec : `401` | 12 k |
| 2 | `read_file: login.spec.ts` | 80 lignes | 14 k |
| 3 | `grep: "createSession"` | 3 occurrences | 15 k |
| 4 | `read_file: session.ts` | 210 lignes | 19 k |
| 5 | `edit: session.ts` | diff appliqué, après accord | 20 k |
| 6 | `bash: npm test -- login` | succès | 21 k |
| 7 | *pas d'outil* : réponse finale | « Corrigé » | 21 k |

© ZENIKA 2026 | All rights reserved

# Ce que coûte cette trace




122 k tokens d'entrée envoyés au total, dont 83 % lus en cache

**~10 %**

**le prix du préfixe lu en cache**

Chaque tour ne rajoute que quelques milliers de tokens.

© ZENIKA 2026 | All rights reserved

# Comment un agent déraille



**Boucle sans fin**

→ limite de tours



**Dérive de contexte**

→ sessions courtes



**Faux succès**

→ revue du diff



**Outil halluciné**

→ schémas stricts



**Explosion de coût**

→ budget par tâche



**Injection indirecte**

→ partie 5

© ZENIKA 2026 | All rights reserved

Partie 4 · 20 min

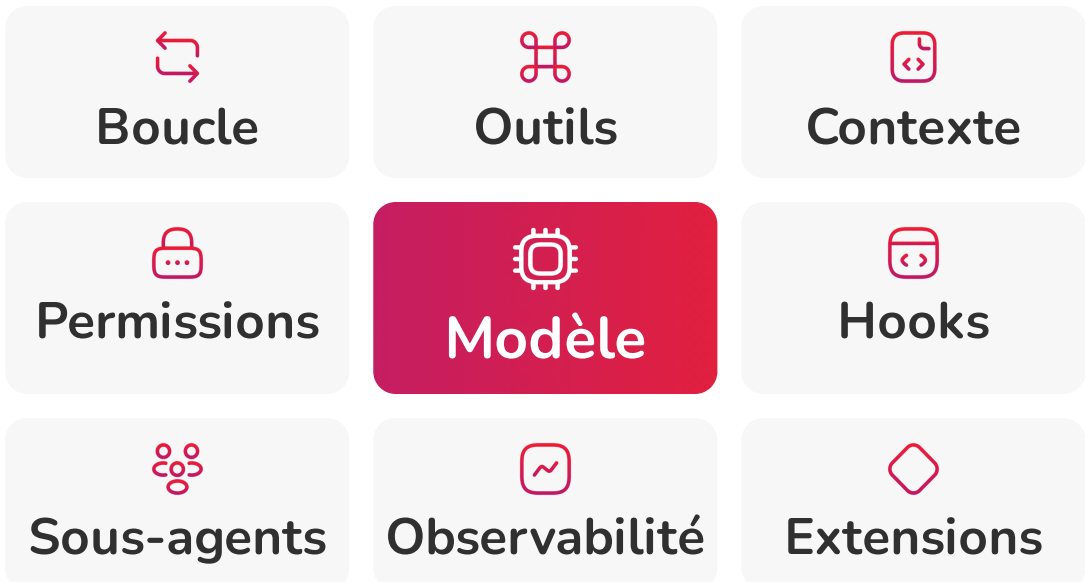
# Harness & orchestrateurs

Le produit qui implémente la boucle, puis ce qu'on ajoute quand un seul agent ne suffit plus.




© ZENIKA 2026 | All rights reserved

# Anatomie d’un harness




© ZENIKA 2026 | All rights reserved

# Le marché des harness

DÉTAIL ↗

ÉDITEUR DU MODÈLE

## Intégrés verticalement

Claude Code · Codex · Gemini CLI

PLATEFORME DEV

## Intégrés à l'outillage

GitHub Copilot · Cursor · Kiro

MODÈLE AU CHOIX

## Open source

OpenCode · Aider · Cline

CONSTRUIRE LE VÔTRE

## SDK d'agents

Claude Agent SDK · OpenAI Agents SDK · Google ADK

UNE ISSUE → UNE PR

## Agents cloud

Copilot agent · Devin · Jules

© ZENIKA 2026 | All rights reserved

# Les cinq patterns de base

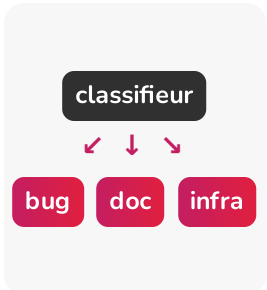
## Chaîne




étapes fixes, chaque sortie nourrit la suivante

> *spec → plan → code*

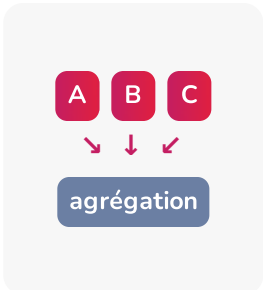
## Routeur




on oriente vers le bon agent ou le bon modèle

> *un SLM trie, un LLM traite*

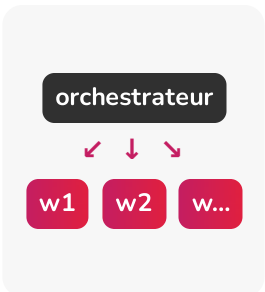
## Parallélisation




tâches indépendantes, ou vote entre plusieurs

> *3 revues d'une même PR*

## Orchestrateur-workers




le découpage est décidé dynamiquement par un LLM

> *refacto sur 12 modules*

## Évaluateur




on itère jusqu'à ce que le critère soit atteint

> *code + revue par un 2e agent*

*workflow figé : chemin connu à l'avance*

*workflow dynamique : le LLM pilote davantage*

© ZENIKA 2026 | All rights reserved

# Au-delà des cinq : quatre patterns de plus

## Agent juge




un agent outillé vérifie : il lance les tests, lit le diff

> *valider une PR avant l'humain*

## Adversarial




un agent cherche la faille, l'autre défend, un arbitre tranche

> *red team : des tests qui cassent le code*

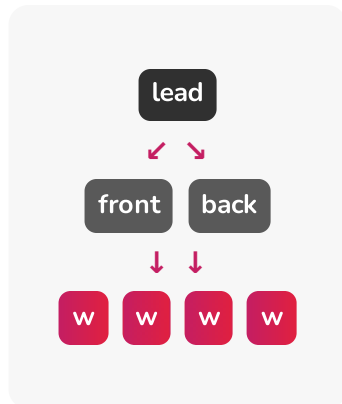
## Handoff




l'agent transmet la conversation et le contexte à un autre

> *support : triage → facturation*

## Hiérarchique



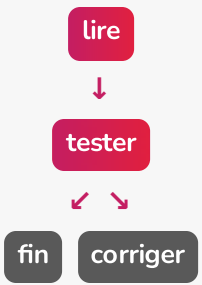

des orchestrateurs d'orchestrateurs, une équipe par domaine

> *une équipe d'agents par service*

© ZENIKA 2026 | All rights reserved

# Qui trace le chemin ?

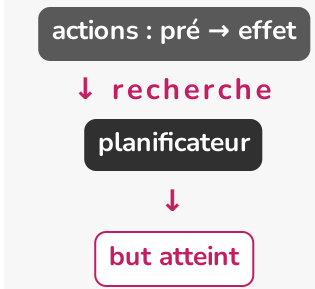
## Graphe




le développeur, à l'avance : nœuds et transitions

> *LangGraph, n8n*

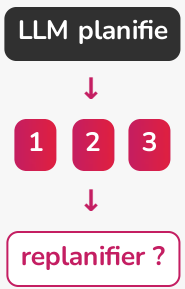
## GOAP




un algorithme, à partir du but et de l'état

> *jeu vidéo (F.E.A.R.), Embabel*

## Plan & exécute




le modèle, au départ, puis il exécute

> *mode plan de Claude Code*

## ReAct




le modèle, à chaque tour, sans plan

> *la boucle de la partie 3*

*chemin fixé : prévisible, testable*

*chemin improvisé : flexible, imprévisible*

© ZENIKA 2026 | All rights reserved

# Le marché des orchestrateurs

DÉTAIL ↗

### FRAMEWORKS (CODE)

**On décrit le graphe**

LangGraph · CrewAI · MS Agent Framework

### EXÉCUTION DURABLE

**Ne jamais perdre une tâche**

Temporal · Inngest · Restate

### CLOUD MANAGÉ

**Agents hébergés**

Bedrock AgentCore · Vertex AI · Azure AI Foundry

### LOW-CODE

**Glisser-déposer**

n8n · Dify · Langflow

### DANS LE HARNESS

**Souvent, ça suffit**

sous-agents · worktrees parallèles

© ZENIKA 2026 | All rights reserved

# Et le SDD dans tout ça ?

1



## Spécifier

Le quoi et le pourquoi.

ARTEFACT

spec.md

2



## Planifier

Le comment.

ARTEFACT

plan.md

3



## Découper

De petites tâches.

ARTEFACT

tasks.md

4



## Implémenter

Un agent par tâche.

ARTEFACT

code + tests

© ZENIKA 2026 | All rights reserved

Partie 5 · 15 min

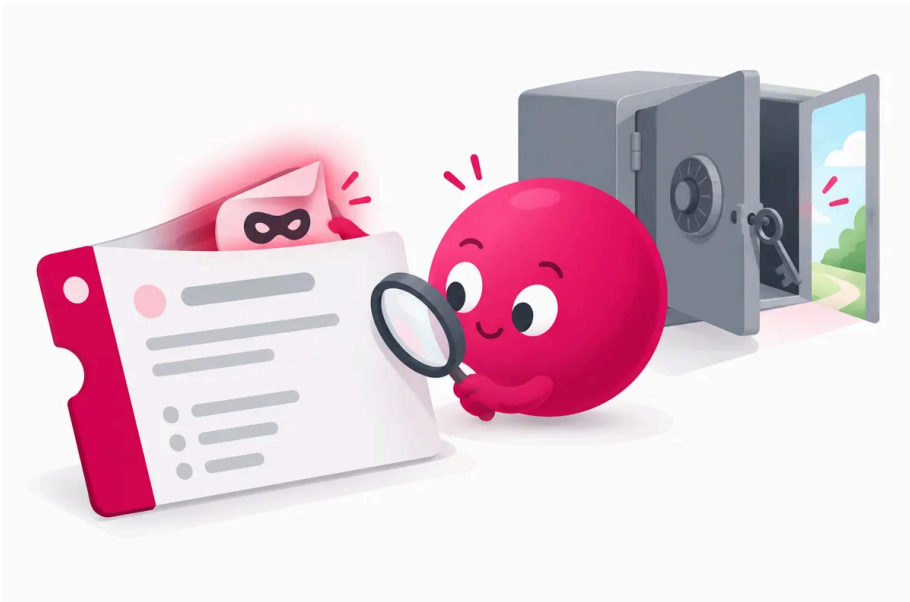
# Guardrails & gateways

Des agents qui lisent, exécutent et appellent des API : comment l'entreprise garde le contrôle ?




© ZENIKA 2026 | All rights reserved

# Ce dont on se protège



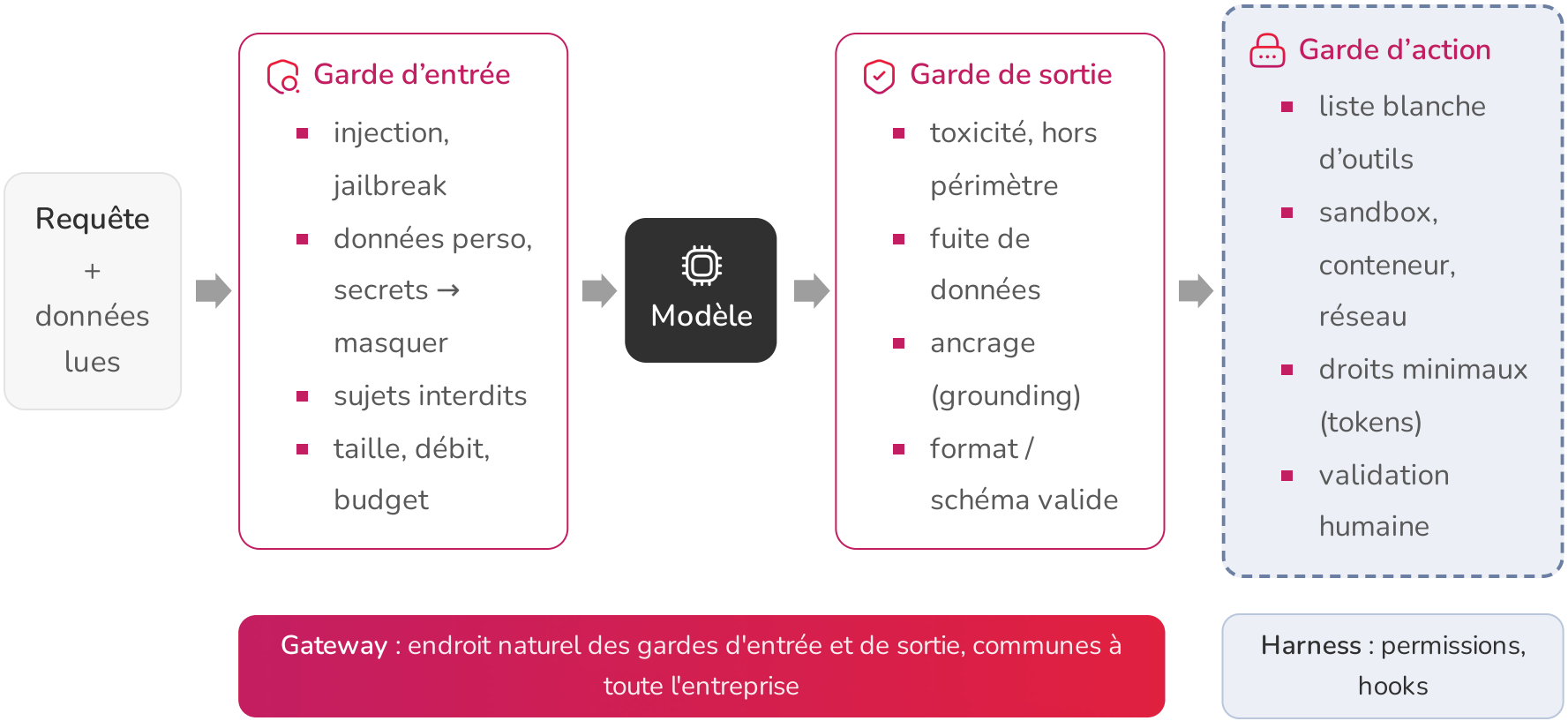
MENACE N° 1 · OWASP LLM01

## Injection de prompt : la consigne est cachée dans ce que l'agent lit.

- Fuite de données
- Agentivité excessive
- Supply chain
- Coût non borné

© ZENIKA 2026 | All rights reserved

# Où placer les contrôles




© ZENIKA 2026 | All rights reserved

# Les fournisseurs de guardrails

DÉTAIL ↗

### SERVICE MANAGÉ

## Chez votre cloud

- Bedrock Guardrails
- Azure Content Safety
- Model Armor

### OPEN SOURCE

## Auto-hébergé

- NeMo Guardrails
- Guardrails AI
- Llama Guard

### API ET SPÉCIALISTES

## Prêt à brancher

- Lakera Guard
- OpenAI Moderation
- Mistral Moderation

© ZENIKA 2026 | All rights reserved

# La gateway IA : un seul point de passage



*le péage : on y passe tous, on y contrôle, on y compte*

**Routage** | **Budgets** | **Cache**

**Logs** | **Guardrails** | **Conformité**

© ZENIKA 2026 | All rights reserved

# Les fournisseurs de gateways

DÉTAIL ↗

OPEN SOURCE

## Auto-héberger

LiteLLM

Envoy AI Gateway

VOTRE API GATEWAY

## Son extension IA

Kong AI Gateway

Azure APIM

SAAS

## Démarrer vite

Portkey

Cloudflare AI Gateway

OpenRouter

© ZENIKA 2026 | All rights reserved

# Une requête, de bout en bout

POUR ALLER PLUS LOIN ↗

1



**SDD**
Une tâche issue d'une spec relue.

2



**Orchestration**
La confie à un agent.

3



**Harness**
Contexte, préfixe stable en tête.

4



**Gateway**
Filtre, route, journalise.

5



**Modèle**
Tokenise, prefill en cache, decode.

6



**Harness**
Permission, outil, reboucle.

**Le modèle prédit, le harness agit, la gateway gouverne.**

© ZENIKA 2026 | All rights reserved

# Des questions ?

Le modèle prédit, le harness agit, la gateway gouverne.

© ZENIKA 2026 | All rights reserved